

Arithmetic Operators in SQL Server with Examples

In this article, I am going to discuss the **Arithmetic operators in SQL Server** with Examples. Please read our previous article where we discussed the [Assignment Operator](#) with examples. At the end of this article, you will understand what are arithmetic operators and how to use different types of arithmetic operators in SQL Server with examples.

What are Arithmetic Operators in SQL Server?

Arithmetic operators are used to performing mathematical calculations such as Addition, Subtraction, Multiplication, and Division on the given operand values. That means these operators are used to perform mathematical operations on two expressions of the same or different data types of numeric data. These different arithmetic operators are as follows:

1. **+** (Addition Operator)
2. **-** (Minus Operator)
3. ***** (Multiplication Operator)
4. **/** (Division Operator)
5. **%** (Modulo Operator)

Arithmetic Operators Example:

```
SELECT 100 + 200 O/P = 300
SELECT 45 - 74 O/P = -29
SELECT 25 * 4 O/P = 100
SELECT 36 / 6 O/P = 6
SELECT 17 % 4 O/P = 1
```

Similar to basic arithmetic calculations, arithmetic operators in SQL Server also have the Operator Precedence.

How arithmetic operators are evaluated in SQL Server?

If the arithmetic expression contains more than one operator, the multiplication operator, and division operator are evaluated first, and then the addition and minus operator are evaluated. When two operators have the same priority, the expression is evaluated from left to right. Parentheses can be used to force an operation to take priority over any other operators. Parentheses are also used to improve code readability.

Arithmetic Operators with complex queries:

We may not always want to retrieve the data as it is. In some scenarios, we may want to display our data with certain calculations. Let us understand how to use arithmetic operators with such complex SQL Queries. We are going to use the following Employee table to understand this.

ID	Name	EmailID	Gender	Department	Salary	Age	CITY
1	Pranaya	Pranaya@g.com	Male	IT	25000	30	Mumbai
2	Tarun	Tarun@g.com	Male	Payroll	30000	27	Odisha
3	Priyanka	Priyanka@g.com	Female	IT	27000	25	Bangalore
4	Preety	Preety@g.com	Female	HR	35000	26	Bangalore
5	Ramesh	Ramesh@g.com	Male	IT	26000	27	Mumbai
6	Pramod	Pramod@g.com	Male	HR	29000	28	Odisha
7	Anurag	Anurag@g.com	Male	Payroll	27000	26	Odisha
8	Hina	Hina@g.com	Female	HR	26000	30	Mumbai
9	Sambit	Sambit@g.com	Male	Payroll	30000	25	Odisha
10	Manoj	Manoj@g.com	Male	HR	30000	28	Odisha
11	Sara	Sara@g.com	Female	Payroll	28000	27	Mumbai
12	Lima	Lima@g.com	Female	HR	30000	30	Bangalore
13	Dipak	Dipak@g.com	Male	Payroll	32000	25	Bangalore

Please use the below script to create and populate the Employee table with the required data.

```
--Create database EmployeeDB
Create database EmployeeDB
Go
Use EmployeeDB
Go
--Create Employee table
CREATE TABLE Employee
(
    ID INT PRIMARY KEY IDENTITY(1,1),
    Name VARCHAR(100),
    EmailID VARCHAR(100),
    Gender VARCHAR(100),
    Department VARCHAR(100),
    Salary INT,
    Age INT,
    CITY VARCHAR(100)
)
GO
--Insert some data into Employee table
INSERT INTO Employee VALUES('Pranaya','Pranaya@g.com','Male','IT',25000,30,
INSERT INTO Employee VALUES('Tarun','Tarun@g.com','Male','Payroll',30000,27,
INSERT INTO Employee VALUES('Priyanka','Priyanka@g.com','Female','IT',27000,25,'Bangalore')
INSERT INTO Employee VALUES('Preety','Preety@g.com','Female','HR',35000,26,'Bangalore')
INSERT INTO Employee VALUES('Ramesh','Ramesh@g.com','Male','IT',26000,27,'M'
```

```

INSERT INTO Employee VALUES('Pramod','Pramod@g.com','Male','HR', 29000, 28,'Odisha')
INSERT INTO Employee VALUES('Anurag','Anurag@g.com','Male','Payroll', 27000, 26,'Odisha')
INSERT INTO Employee VALUES('Hina','Hina@g.com','Female','HR', 26000, 30,'Mumbai')
INSERT INTO Employee VALUES('Sambit','Sambit@g.com','Male','Payroll', 30000, 28,'Odisha')
INSERT INTO Employee VALUES('Manoj','Manoj@g.com','Male','HR', 30000, 28,'Odisha')
INSERT INTO Employee VALUES('Sara','Sara@g.com','Female','Payroll', 28000, 25,'Bangalore')
INSERT INTO Employee VALUES('Lima','Lima@g.com','Female','HR', 30000, 30,'Bangalore')
INSERT INTO Employee VALUES('Dipak','Dipak@g.com','Male','Payroll', 32000, 25,'Bangalore')
GO

```

Example: Arithmetic Operators in SQL Server

The below SQL Server example calculates the employee's salary after the addition of 10%.

```

SELECT ID, Name, Department, Salary,
       Salary * 1.1 AS CalculatedSalary
FROM   Employee

```

It is always advisable to specify the original column without any change (salary) in addition to the column with the calculation (salary * 1.1) in order to make the difference. It is also possible to perform any mathematical calculation as shown in the below example.

```

SELECT ID, Name, Department, Salary,
       Salary * 0.11 / 2.54 + 27.36 AS CalculatedSalary
FROM   Employee

```

If we want to provide the order precedence then simply use round brackets as shown in the below example

```

SELECT ID, Name, Department, Salary,
       Salary * 0.11 / (2.54 + 27.36) AS CalculatedSalary
FROM   Employee

```

Note: SQL Server syntax ignores blank spaces before and after the arithmetic operator. Along with, in SQL Server we can also use arithmetic operators on multiple columns:

Arithmetic Operations on String Values in SQL Server

In SQL Server it is not possible to perform any calculations on String values (such as Varchar or Char). The following SQL statement will give you an error.

```
SELECT ID, Name + 100 FROM Employee
```

Error: Conversion failed when converting the varchar value 'Some String Value' to data type int.

Arithmetic Operations on Date Values

In SQL Server the plus (+) and minus (-) operators can be used to perform arithmetic operations on Date values. These operators will allow you to add or subtract days.

SELECT GETDATE() + 1 Gives Current date + 1 day

SELECT GETDATE() - 1 Gives Current date - 1 day

In SQL Server It is not possible to use Multiplication or Division on Date values. The following examples will give you errors.

SELECT GETDATE() * 5

SELECT GETDATE() / 5

Error: Implicit conversion from data type DateTime to int is not allowed. Use the CONVERT function to run this query.

Note: In SQL Server a NULL value is a value that indicates an empty field in a table. This value does not equal zero (0), nor does it equals space (' '); When we performed a certain calculation on a column that contains NULL values, then it returns NULL.

In the next article, I am going to discuss the [Comparison Operators in SQL Server](#). Here, in this article, I try to explain the **Arithmetic Operators in SQL Server** with Examples. I hope you enjoy this article.